Hooking into the **component lifecycle**

https://angular.io/guide/lifecycle-hooks

A **component instance** has a **lifecycle** that
starts when **Angular instantiates** the **component class** and
renders **the component view** along with its **child views**.

The **lifecycle**
continues with **change detection**,
as Angular **checks** to see when **data-bound properties change**, and
updates both the **view** and the **component instance** as needed.

The **lifecycle ends** when
Angular **destroys** the **component instance** and
removes its **rendered template** from the **DOM**.

Directives have a **similar lifecycle**,
as Angular creates, updates, and destroys instances in
the course of execution.

The application can use
lifecycle hook methods to
tap into **key events** in
the **lifecycle** of
a **component** or **directive**
in order to
initialize new **instances**,
initiate change detection,
respond to **updates** during **change detection**, and
clean up before
deletion of **instances**.

You can respond to events in the lifecycle of a component or directive by implementing **one** or **more** of the **lifecycle hook interfaces** in the Angular core library.

The hooks give you the opportunity to act on
a **component** or **directive instance** at the appropriate moment,
as Angular **creates**, **updates**, or **destroys** that instance.

Each **interface** defines the **prototype** for a **single hook** method,
whose name is the interface name prefixed with ng.
For example, the **OnInit** interface has a hook method named **ngOnInit()**.
If you implement this method in your **component** or **directive** class,
Angular calls it shortly **after checking** the **input properties** for
that **component** or **directive** for the **first time**.

```
@Directive()
export class PeekABooDirective implements OnInit {
  constructor(private logger: LoggerService) { }

  // implement OnInit's `ngOnInit` method
  ngOnInit() {
    this.logIt(`OnInit`);
  }

  logIt(msg: string) {
    this.logger.log(`#${nextId++} ${msg}`);
  }
}
```

Lifecycle
Event Sequence

After your application instantiates
a component or directive by calling its constructor,
Angular calls the hook methods you have implemented at
the appropriate point in the lifecycle of that instance.

Angular executes hook methods in the following sequence.
You can use them to perform the following kinds of operations.

Angular calls these hook methods in
the following order:

1. ngOnChanges:

When an **input / output binding** value **changes.**
When Angular **sets** or **resets data-bound input properties**.
The method receives a **SimpleChanges** object of
current and **previous property values**.

See details in Using change detection hooks

```
ngOnChanges(changes: SimpleChanges) {
  for (const propName in changes) {
    const chng = changes[propName];
    const cur  = JSON.stringify(chng.currentValue);
    const prev = JSON.stringify(chng.previousValue);
    this.changeLog.push(`${propName}: currentValue = ${cur}, previousValue = ${prev}`);
  }
}
```

Called **before ngOnInit()** and
whenever one or **more**
data-bound input properties **change**.
Note that if your component has **no inputs** or
you use it **without** providing any **inputs**,
the framework **will not call ngOnChanges().**

2. ngOnInit:
After the first **ngOnChanges.**

A **lifecycle hook** that is called
after Angular has **initialized**
all data-bound properties of a **directive.**

Called **once**, after the **first ngOnChanges()**.

Initialize the **directive** or **component**
after Angular **first** displays the **data-bound properties** and
sets the **directive** or **component's input properties**.
See details Initializing a component or directive

ДОВЪРШИ ОТ 3 НАДОЛУ
https://angular.io/guide/lifecycle-hooks

3. ngDoCheck:
Developer's custom change detection

4. ngAfterContentInit:
After component content **initialized**

5. ngAfterContentChecked:
After every **check** of component content

6. ngAfterViewInit:
After a component's **views are initialized**

@ViewChild
По-добре виж всички линкове всичко е там, тук не е пълно:
https://dev.to/danielpdev/how-to-use-viewchild-decorator-in-angular-9-i0

Using **@ViewChild** decorator
Какъв е scope на заявките за шаблон @ViewChild?

6.1 Using **@ViewChild** to **inject**
a **reference** to a component:

SomeComponent

```
<h2>Choose Brand Colors:</h2>

<color-sample [color]="primary" #primaryColorSample>
</color-sample>

<mat-input-container>
  <mat-label>Primary Color</mat-label>
  <input matInput #primaryInput [(colorPicker)]="primary" [(ngModel)]="primary"/>
</mat-input-container>
```

AppComponent

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent {
  .....

  @ViewChild(ColorSampleComponent)
  primarySampleComponent: ColorSampleComponent;

  .....
}
```

This injected **ColorSampleComponent** instance is the same one linked to the **<color-sample>** custom element present.
ColorSampleComponent е **ColorSample** + **Component** от **<color-sample>**
Това не са визуализира веднага, става по-късно в component lifecycle, **after** the **view initialization** е завършила

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent implements  AfterViewInit {
  .....

  @ViewChild(ColorSampleComponent)
  primarySampleComponent: ColorSampleComponent;

  ngAfterViewInit() {
    console.log('Values on ngAfterViewInit():');
    console.log("primaryColorSample:", this.primarySampleComponent);
  }

  .....
}
```

С **@ViewChild** можем да инжектираме всеки компонент или директива (или HTML елемент), присъстващи в html template-а на компонента в

който слагаме **@ViewChild.**

Но докъде можем да query компоненти в дървото на компонентите?

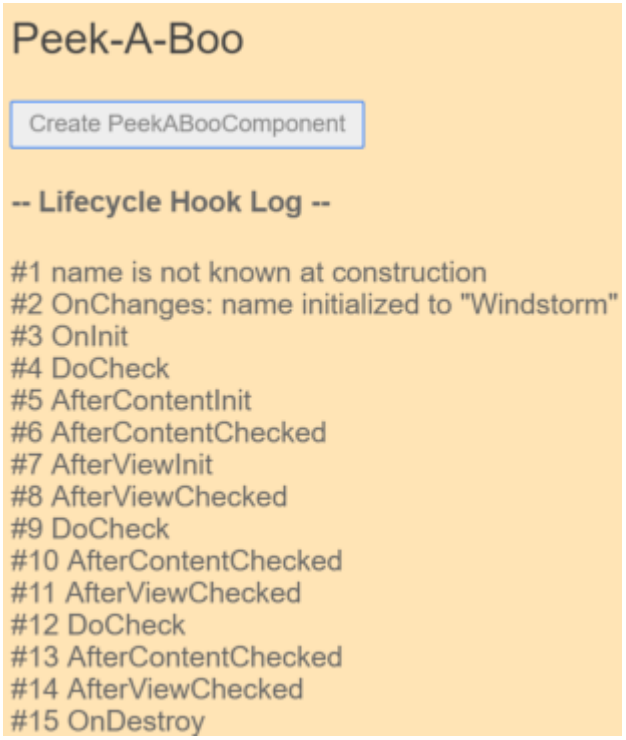
Използваме **@ViewChild** за query за компонент, който е по -дълбоко в дървото на компонентите.
Като пример, нека да разгледаме компонента **<color-sample>:**

За да обобщим: декораторът @ViewChild
е механизъм за заявки по шаблон,
който е локален за компонента.
https://blog.angular-university.io/angular-viewchild/

7. ngAfterViewChecked:
After every **check** of a **component's views**

8. ngOnDestroy:
Just **before** the directive is **destroyed**

The following snapshot
reflects the state of the log
after the user clicked the *Create...* button and
then the *Destroy...* button.



Ако потребителят щракне върху бутона Update Hero
the log would show another:

OnChanges and
DoCheck,
AfterContentChecked and
AfterViewChecked.

Notice that these three hooks fire often,
so it is important to keep their logic as lean as possible.

ViewChild decorator

https://blog.angular-university.io/angular-viewchild/